



Programming

Recursion

Markos Vassilis, Mediterranean College

Week 11

A Crash Course on Recursion

What Will This Print?

```
def foo(n):  
    if n == 0:  
        return True  
    if n == 1:  
        return False  
    return not foo(n - 1)  
  
print({x: foo(x) for x in range(10)})
```

Expected Output

```
{0: True, 1: False, 2: True, 3: False, 4: True, 5: False, 6: True, 7: False, 8: True, 9: False}
```

- Why is that?
- Can you explain how `foo( )` works?
- How can we actually call a function from within its own definition? Why does this even terminate?
- This is what we call **Function Recursion!**

Recursion

- Recursion has (at least) two stages:
 - The base case, in which recursion terminates (e.g., the if condition in our previous example).
 - The recursive case(s), in which we call the function itself with different arguments.
- Every loop can be implemented using recursion.
- Recursion includes only function calls and some if statements.
 - if all these function calls are to immutable functions (or functions with no side effects), and;
 - since if statements can be written in an immutable manner;
 - then, recursion provides a way to represent loops immutably.

Recursive Sums

The following Python function computes the sum of a list's elements (numbers):

```
def simple_sum(numbers):  
    s = 0  
    for x in numbers:  
        s += x  
    return s  
  
if __name__ == "__main__":  
    numbers = [5, 0, 3, -2]  
    print(simple_sum(numbers))
```

Recursive Sums

How about recursion?

```
def rec_sum(numbers):  
    if len(numbers) == 0:  
        return 0  
    return numbers[0] + rec_sum(numbers[1:])
```

Essentially, we just sum the elements of the list one by one, resolving to the base case of returning 0 whenever the list is empty.

Product of Numbers

Write a recursive function that accepts a list and computes the product of its elements.

```
def rec_prod(numbers):  
    if len(numbers) == 0:  
        return 1  
    return numbers[0] * rec_sum(numbers[1:])
```

Recursive Power

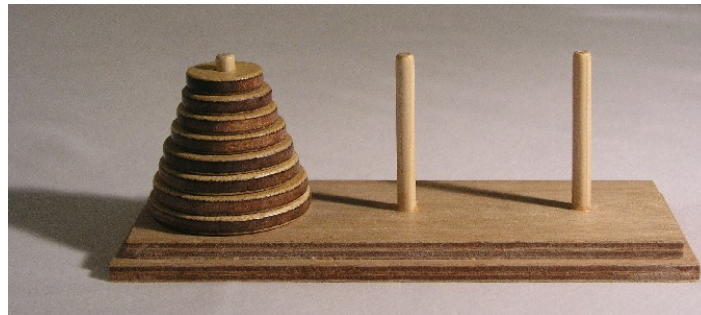
Write a Python Recursive function that accepts two **integer** arguments, a and b, and computes **a to the power of b** ($a^{**}b$):

```
def rec_power(a, b):  
    if b == 0:  
        return 1  
    return a * rec_power(a, b - 1)  
  
if __name__ == "__main__":  
    print(rec_power(4, 3))
```

Towers of Hanoi

This is a classic puzzle involving three towers (pegs) and a set of disks of different sizes. The objective is to move all the disks from one peg to another, following these rules:

- Only one disk can be moved at a time.
- A larger disk cannot be placed on top of a smaller one.



A Line of Attack

Consider the following (name the pegs A, B, C, right to left):

- For $n = 1$ disk you just need to move that one disk from A to C.
- For $n = 2$ disks you first move disk 1 from A to B, then disk 2 from A to C and then disk 1 from B to C.
 - This is essentially moving disk 1 to B and then solving the problem for $n-1$ ($=1$) twice.
- For $n = 3$ disks you first solve the $n = 2$ case by moving disks 1 and 2 to B, then move disk 3 to C and solve again the $n = 2$ case by moving disks 1 and 2 to C, on top of disk 3.

A Line of Attack

- For $n = 4$ disks you:
 - First move disks 1, 2, 3 to B, solving the $n = 3$ case.
 - Then move disk 4 to C.
 - Then move disks 1, 2, 3 to C, solving again the $n = 3$ case.
- For $n = 5$ disks you:
 - First move disks 1, 2, 3, 4 to B, solving the $n = 4$ case.
 - Then move disk 5 to C.
 - Then move disks 1, 2, 3, 4 to C, solving again the $n = 4$ case.
- Can you see a pattern here?

A Line of Attack

In general, in order to solve the problem for any n :

- First move the first $n - 1$ disks from A to B (the auxiliary peg).
- Then move disk n to the target peg, C.
- Then move the first $n - 1$ disks from B to C.

The above has a clearly recursive structure, as while solving the case for n we have to solve the case for $n - 1$.

Recursion of Hanoi?

Can you think of a recursive function that solve the Towers of Hanoi problem?

```
def tower_of_hanoi(n, source, destination, auxiliary):  
    if n == 1:  
        print(f"Move disk 1 from {source} to  
{destination}")  
        return  
    tower_of_hanoi(n-1, source, auxiliary, destination)  
    print(f"Move disk {n} from {source} to {destination}")  
    tower_of_hanoi(n-1, auxiliary, destination, source)  
  
if __name__ == "__main__":  
    tower_of_hanoi(3, "A", "C", "B")
```

Complexity of Hanoi Towers

Can you compute the computational complexity class, in terms of Big-O notation for the above function?

- For n disks we need to solve the case for $n-1$ twice and make an additional computation each time.
- Thus, if $f(n)$ is the time complexity of this function, the following holds:

$$f(n) = f(n-1) + 1 + f(n-1) \text{ or } f(n) = 2f(n-1) + 1$$

- So, what is $f(n)$?

Complexity of Hanoi Towers

- Since $f(1) = 1$ (we need one move to solve the $n = 1$ case) we can easily compute:
 - $f(2) = 2f(1) + 1 = 3$
 - $f(3) = 2f(2) + 1 = 7$
 - $f(4) = 2f(3) + 1 = 15$
 - $f(5) = 2f(4) + 1 = 31$
 - $f(6) = 2f(5) + 1 = 63$
 - $f(7) = 2f(6) + 1 = 127$
- Can you guess $f(n)$?
- $f(n) = 2^n - 1$ (can you prove it formally using induction?)

Fibonacci Numbers

Fibonacci numbers are defined by the following relationship:

$$f(n) = f(n-1) + f(n-2),$$

where $f(0) = 0$ and $f(1) = 1$.

Thus, the first Fibonacci numbers are:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, ...

How can we compute the n -th Fibonacci number?

A Piece of Cake

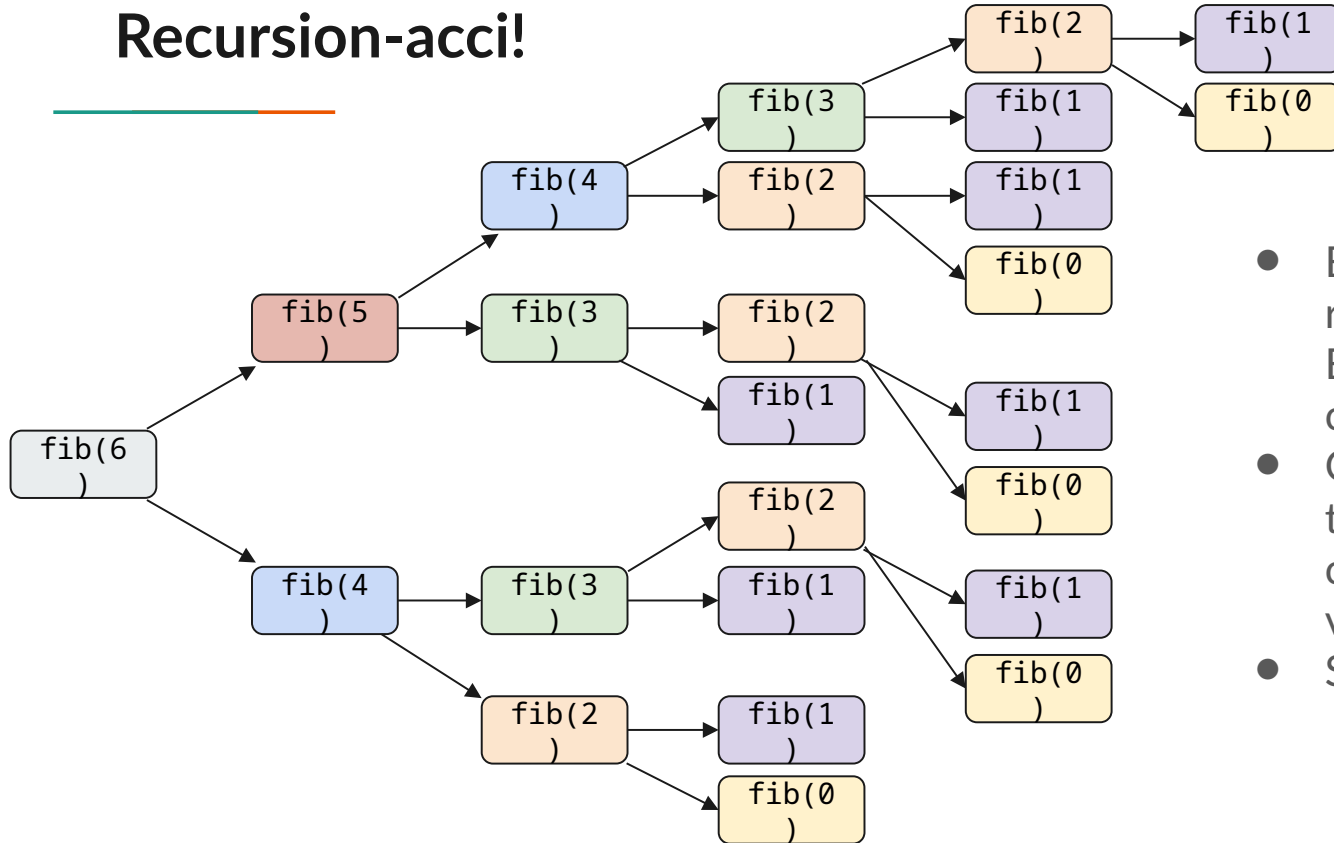
Well, this is some more or less evident chance to flex our recursion muscle:

```
def fib(n):  
    if n == 0:  
        return 0  
    if n == 1:  
        return 1  
    return fib(n - 1) + fib(n - 2)  
  
if __name__ == "__main__":  
    print([fib(n) for n in range(0,20)])
```

Large Fibonacci Numbers

- Which is the 35th President of the US?
 - John Fitzgerald Kennedy
- Which is the 35th Fibonacci number?
 - Use our freshly developed machinery to find out!
- Which is the 36th Fibonacci number?
- Which is the 37th Fibonacci number?
- Which is the 40th Fibonacci number?
- Which is the 50th Fibonacci number?
- Which is the 100th Fibonacci number?
- What has gone wrong here?

Recursion-acci!



- Each function makes 2 recursive calls → Exponential complexity, $O(2^n)$.
- Observe how many times we end up computing the same values $f(n)$.
- Something better?

Simple Yet Fast

Consider the following script. What is its time complexity?

```
def fib(n):  
    if n in [0,1]:  
        return n  
    a = 0, b = 1  
    for i = 1, ..., n-1:  
        c = a + b  
        a = b  
        b = c  
    return c
```

Worst case complexity?

Simple Yet Fast

Consider the following script. What is its time complexity?

```
def fib(n):  
    if n in [0,1]:  
        return n  
    a = 0, b = 1  
    for i = 1, ..., n-1: ← This repeats n-1 times.  
        c = a + b  
        a = b  
        b = c  
    return c
```

Worst case complexity: $O(n)$

So, Is Recursion Slow?

There are two chances here:

- **Recursion is not always the best way** to implement things, which means that Function Programming has more worth as a theoretical construct for mathematicians than it has for actual developers.
- **Recursion is always the best way** to implement things - or, at least, equally best to imperatives - but we really created a mess with our recursive Fibonacci function.

Which do you think is the case?

Better Recursion

Let us revisit our first recursive sum example:

```
def rec_sum(numbers):  
    if len(numbers) == 0:  
        return 0  
    return numbers[0] + rec_sum(numbers[1:])
```

How is evaluation done in this case, say for a list like `numbers = [1, 2, 3, 4]`?

Better Recursion

```
rec_sum([1, 2, 3, 4]
1 + rec_sum([2, 3, 4])
1 + (2 + rec_sum([3, 4]))
1 + (2 + (3 + rec_sum([4])))
1 + (2 + (3 + (4 + (rec_sum([])))))
1 + (2 + (3 + (4 + 0))
1 + (2 + (3 + 4))
1 + (2 + 7)
1 + 9
10
```

Better Recursion

The above execution:

- Is really complex for larger values of n .
- Requires n function calls to complete before evaluation.
- Most importantly, relies on the size of the memory's stack to run for larger values of n (remember, usually the stack is much smaller than the heap in memory).
- So, this way of writing recursion is actually quite restrictive.
- But, what could we do, instead?

Tail Recursion

Consider this alternative recursive sum function:

```
def tc_sum(numbers, temp_sum = 0):  
    if len(numbers) == 0:  
        return temp_sum  
    return tc_sum(numbers[1:], temp_sum + numbers[0])  
  
if __name__ == "__main__":  
    a = [5, 2, 8, 6, -4, 6]  
    print(tc_sum(a))
```

What's the difference here?

Tail Recursion

In this case, function calls are evaluated as follows (for input [1, 2, 3, 4]):

```
tc_sum([1, 2, 3, 4], 0)
tc_sum([2, 3, 4], 1)
tc_sum([3, 4], 3)
tc_sum([4], 6)
tc_sum([], 10)
10
```

Observe how in this case only the last call needs to complete, as it returns the actual value! This seems to save a lot of memory!

Tail Call Optimisation (TCO)

- While seeming amazingly faster, the previous implementation has as a prerequisite that the implementation of a programming language is **clever enough** to recognise and optimise functions returning calls to other functions.
- Such an optimisation is called **Tail Call Optimisation**.
- Essentially, this optimisation boils down to avoiding opening multiple windows in the memory's stack, effectively making a tail recursive function generating a constant (and small) number of frames in the stack.
- This is implemented simply by adding a jump (GOTO) during compilation at the return statement of such functions.

Tail Call Optimisation (TCO)

- However, adding such jumps has its drawbacks.
- The most significant could be that this makes it difficult to traceback errors and function chains, in general.
- Thus, one has to make a decision on whether to support TCO or not.
- **Python has made its call, which is not to support TCO.**
- That is, tail recursion will be of no benefit to us when using Python. However, we shall use it mostly for demonstration purposes.
- **JavaScript**, on the other hand (since ECMAScript 6) **supports TCO.**

Tail Factorial

Recall our recursive function for the factorial of an integer from last lecture:

```
def rec_factorial(n):  
    if n == 0:  
        return 1  
    return n * rec_factorial(n - 1)  
  
if __name__ == "__main__":  
    print(rec_factorial(5))
```

Write a **tail recursive** version of the above!

Tail Factorial

A tail recursive version of the above might be as follows:

```
def tail_factorial(n, current_factorial = 1):  
    if n == 0:  
        return current_factorial  
    return tail_factorial(n - 1, current_factorial * n)  
  
if __name__ == "__main__":  
    print(tail_factorial(6))
```

Fibonacci Revisited

So, how can we improve the following to build a tail recursive function that computes the n-th Fibonacci number?

```
def fib(n):  
    if n == 0:  
        return 0  
    if n == 1:  
        return 1  
    return fib(n - 1) + fib(n - 2)
```

Tail Fibonacci

Again, the trick here is to keep track of the “current” version of the recursive output, as done below:

```
def tail_fib(n, current_fib = 1, prev = 0):  
    if n == 0:  
        return current_fib  
    return tail_fib(n - 1, current_fib + prev, current_fib)  
  
if __name__ == "__main__":  
    print([tail_fib(n) for n in range(1, 20)])
```

Epilogue: Lambda Functions

Lambda functions are a key feature of lambda calculus, as you may well guess by their name.

- Python, as with several other languages, provides support for such functions.
- The syntax is quite simple, as shown below:

```
foo = lambda x : x + 1
```

- This defines a function stored at a variable, `foo`, which accepts a single parameter `x` as input and returns `x + 1`.
- Example call: `foo(5)`.

Epilogue: Examples of lambda Functions

- Write a lambda function that takes a string as an argument and returns its last character.

```
bar = lambda x : x[-1]
```

- Write a lambda function that takes two numbers as inputs and computes their sum.

```
foo = lambda x, y : x + y
```

- Write a lambda function that takes a list of numbers as input and computes its average.

```
foo = lambda x : sum(x) / len(x)
```

Fun Time!

You Decide!

You are given two options:

1. Resume working / Complete any unfinished lab exercises from last time on the three revision labs (arrays, files, and OOP).
2. Work with the recursion related exercises that follow (see next slides).

In any case, you have to write some code for today's fun time...

Exercise #001

- Write a recursive function that accepts a string from the user and computes its length.
- Make sure your function is tail recursive!

Exercise #002

Write a recursive list membership function. That is, your function:

- should, provided a list of integers, x , and an integer, a ;
- check whether a is in x or not.

Try to make your function tail recursive!

Exercise #003

Write a recursive function that:

- takes a list of integers as an input, and;
- returns the maximum element of that list.

Make sure your function is tail recursive!

Exercises #004

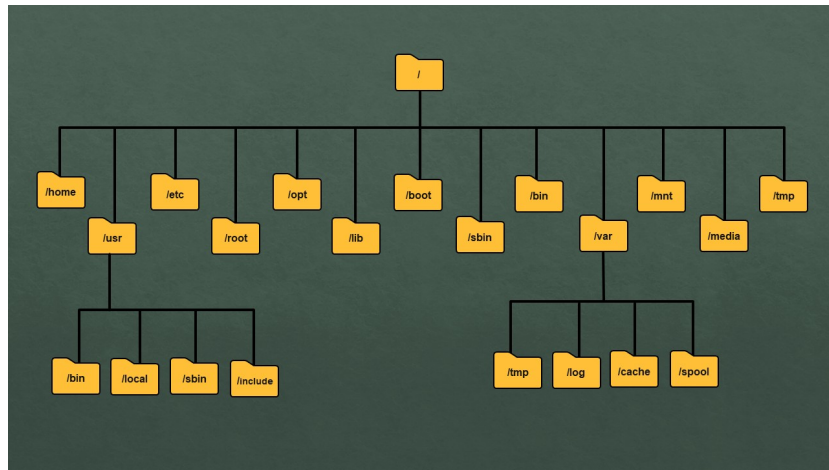
We say that a string is palindrome if reading it left to right and right to left results to the same string.

- Write a recursive Python function that accepts a string as an input and checks whether it is palindrome or not.
- Your function should be tail recursive.

Exercise #005

Directory structure in an Operating System is often represented as a tree (see right).

- Write a Python function that recursively visits and prints all sub-directories of a user provided directory.
- Make sure your function is tail recursive!
- *You can look around the web for how to manipulate directories with Python!*



Exercise #006

Let's play with lambda functions! (never sounds that funny, right?)

- Write a lambda function that takes a list of numbers and finds its maximum element.
- Write a lambda function that takes a list of numbers and finds its second largest element.
- Write a lambda function that takes a list of numbers and finds its second largest element without using any built-in sorting function!
 - You can define some helper functions for this task!

The Actual Fun...

Thinking recursively is a bit confusing, especially as one begins their journey with recursion and functional programming.

As our next lab exercise, follow this tutorial, which is exactly about how to think recursively:

<https://realpython.com/python-thinking-recursively/>

Homework

As your homework:

- Complete any pending in-class exercises.
- Follow the tutorial found below:

<https://python-course.eu/advanced-python/recursive-functions.php>

For any questions, or anything relevant, contact me at: `v.markos@mc-class.gr`

Any Questions

Don't forget to fill in the questionnaire shown right!



<https://forms.gle/YU1mjsxjBjBf8hyvH7>